Corso completo di C++ moderno

Dalle basi al C++20/23: strutture dati, algoritmi, web, mobile, videogiochi, embedded, sistemi operativi, reti e database

Con esercizi consigliati da LeetCode per ogni sezione

Indice

Parte 1 - Fondamenti del linguaggio 4

Capitolo 1 - Introduzione e ambiente di lavoro . . . 4
Capitolo 2 - Tipi, variabili e operatori . . . 5
Capitolo 3 - Controllo di flusso e funzioni . . . 6
Capitolo 4 - Puntatori, riferimenti e memoria . . . 7
Capitolo 5 - Stringhe, I/O e file . . . 7

Parte 2 - OOP, template e C++ moderno 9

Capitolo 6 - Programmazione orientata agli oggetti . . . 9
Capitolo 7 - Template e programmazione generica . . . 10
Capitolo 8 - Il C++ moderno: C++11, 14, 17, 20, 23 . . . 11

Parte 3 - STL e strutture dati 13

Capitolo 9 - La Standard Template Library . . . 13
Capitolo 10 - Strutture dati classiche (implementarle da zero) . . . 14

Parte 4 - Algoritmi principali 18

Capitolo 11 - Analisi e tecniche di base . . . 18
Capitolo 12 - Ordinamento . . . 18
Capitolo 13 - Ricorsione, backtracking, greedy . . . 19
Capitolo 14 - Programmazione dinamica . . . 20
Capitolo 15 - Algoritmi su grafi . . . 21
Capitolo 16 - Stringhe, matematica e bit manipulation . . . 21

Parte 5 - Concorrenza, prestazioni e librerie 23

Capitolo 17 - Concorrenza e parallelismo . . . 23
Capitolo 18 - Prestazioni e ottimizzazione . . . 24
Capitolo 19 - Le principali librerie del C++ . . . 24

Parte 6 - Programmazione web e mobile 26

Capitolo 20 - Programmazione web con C++ . . . 26
Capitolo 21 - Programmazione mobile . . . 27

Parte 7 - Videogiochi e grafica 29

Capitolo 22 - Fondamenti di game programming . . . 29
Capitolo 23 - Librerie e motori . . . 30
Capitolo 24 - IA, rete e ottimizzazione nei giochi . . . 31

Parte 8 - Sistemi embedded 32

Capitolo 25 - Fondamenti dell'embedded con C++ . . . 32
Capitolo 26 - Protocolli, RTOS e sistemi connessi . . . 33

Parte 9 - Sistemi operativi 35

Capitolo 27 - Programmazione di sistema (POSIX/Linux e Windows) . . . 35
Capitolo 28 - Concetti di sistemi operativi . . . 36
Capitolo 29 - Costruire un sistema operativo . . . 36

Parte 10 - Programmazione di reti 38

Capitolo 30 - Fondamenti di networking . . . 38
Capitolo 31 - Server scalabili . . . 38
Capitolo 32 - Algoritmi e progetti di rete . . . 39

Parte 11 - Programmazione di database 41

Capitolo 33 - Usare database da C++ . . . 41
Capitolo 34 - Costruire un database: strutture e algoritmi . . . 42

Parte 12 - Percorso di studio e progetti finali 44

Capitolo 35 - Piano di studio e preparazione ai colloqui . . . 44

Parte 1 - Fondamenti del linguaggio

Capitolo 1 - Introduzione e ambiente di lavoro

Lezione 1.1 - Cos'e' il C++ e perche' impararlo

Il C++ e' un linguaggio compilato, staticamente tipizzato e multi-paradigma (procedurale, orientato agli oggetti, generico, funzionale). Creato da Bjarne Stroustrup nel 1979 come "C with Classes", e' standardizzato da ISO: C++98, C++03, C++11, C++14, C++17, C++20, C++23. Questo corso usa lo stile del **C++ moderno** (da C++11 in poi): niente `new`/`delete` manuali, niente puntatori grezzi proprietari, molto uso della libreria standard.

Il C++ e' usato dove servono prestazioni e controllo: motori di gioco (Unreal), browser (Chrome, Firefox), database (MySQL, MongoDB, ClickHouse), sistemi operativi e driver, firmware embedded, finanza ad alta frequenza, grafica e machine learning (backend di PyTorch, TensorFlow).

- **Zero-cost abstraction**: non paghi per cio' che non usi.
- **Controllo della memoria**: gestione deterministica delle risorse tramite RAII.
- **Ecosistema enorme**: STL, Boost, Qt, Eigen, Asio, SDL e molto altro.

Lezione 1.2 - Compilatori, toolchain e primo programma

Compilatori principali: GCC (`g++`), Clang (`clang++`), MSVC. Per seguire il corso installa GCC 13+ o Clang 17+ e CMake. Il processo di build ha quattro fasi: preprocessore, compilazione, assemblaggio, linking.

```
#include <iostream>

int main() {
    std::cout << "Ciao, mondo!\n";
    return 0;
}
```

```
g++ -std=c++20 -Wall -Wextra -O2 main.cpp -o main
./main
```

Opzioni utili: `-Wall -Wextra -Wpedantic` (avvisi), `-g` (debug), `-O2` (ottimizzazione), `-fsanitize=address,undefined` (sanitizer, indispensabili durante lo sviluppo).

> **Nota**
>
> Compila sempre con gli avvisi attivi e con i sanitizer in debug: individuano la maggior parte degli errori di memoria e di comportamento indefinito.

Lezione 1.3 - Strumenti: CMake, debugger, IDE

CMake e' lo standard de facto per descrivere progetti C++ multipiattaforma.

```
cmake_minimum_required(VERSION 3.20)
project(corso LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(corso src/main.cpp)
target_compile_options(corso PRIVATE -Wall -Wextra)
```

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j
```

Altri strumenti: **gdb/lldb** (debugger), **valgrind**, **clang-format** (formattazione), **clang-tidy** (analisi statica), **Conan/vcpkg** (gestione dipendenze), **Catch2/GoogleTest** (test), IDE come CLion, VS Code (con clangd) e Visual Studio.

Capitolo 2 - Tipi, variabili e operatori

Lezione 2.1 - Tipi fondamentali

Tipi interi: `int`, `long`, `long long`, `short`, `char` e varianti `unsigned`. Per dimensioni garantite usa `<cstdint>`: `int32_t`, `uint64_t`, ecc. Tipi in virgola mobile: `float`, `double`, `long double`. Altri: `bool`, `char`, `wchar_t`, `char8_t` (C++20), `std::byte`.

```
#include <cstdint>
#include <iostream>
int main() {
    int a = 42;
    double pi = 3.14159;
    std::uint8_t byte = 255;
    bool ok = true;
    auto x = 10u;          // deduzione: unsigned int
    int b{7};              // inizializzazione uniforme (vieta narrowing)
    constexpr int N = 100; // valutata a compile time
    std::cout << sizeof(int) << ' ' << sizeof(long long) << '\n';
}
```

Preferisci l'inizializzazione con parentesi graffe `{}`: impedisce le conversioni con perdita (`int x{3.5};` e' un errore). `auto` deduce il tipo, `const` e `constexpr` esprimono immutabilita'.

Esercizi consigliati su LeetCode

7 - Reverse Integer (overflow e limiti dei tipi)

9 - Palindrome Number

13 - Roman to Integer

Lezione 2.2 - Operatori e conversioni

Aritmetici (`+ - * / %`), confronto, logici (`&& || !`), bit a bit (`& | ^ ~ << >>`), assegnazione composta, ternario `?:`. Attenzione a: divisione tra interi (tronca), overflow dei signed (comportamento indefinito), confronti signed/unsigned. Usa `static_cast<T>(x)` per le conversioni esplicite; evita i cast in stile C.

```
int n = 7;
double r = static_cast<double>(n) / 2;   // 3.5
unsigned flags = 0b1010;
flags |= (1u << 0);                      // imposta il bit 0
bool bit3 = flags & (1u << 3);
```

Il C++20 introduce l'operatore a tre vie `<=>` (spaceship) che genera automaticamente tutti i confronti.

Esercizi consigliati su LeetCode

191 - Number of 1 Bits

231 - Power of Two

136 - Single Number (XOR)

338 - Counting Bits

190 - Reverse Bits

Capitolo 3 - Controllo di flusso e funzioni

Lezione 3.1 - Istruzioni condizionali e cicli

`if`, `else`, `switch`, `for`, `while`, `do-while`, `break`, `continue`. Il C++17 consente l'inizializzazione nell'`if` e nello `switch`; il C++11 introduce il ciclo range-based.

```
#include <vector>
#include <map>
#include <iostream>
int main() {
    std::vector<int> v{1, 2, 3, 4};
    for (int x : v) std::cout << x << ' ';

    std::map<std::string, int> m{{"a", 1}, {"b", 2}};
    for (const auto& [chiave, valore] : m)        // structured bindings (C++17)
        std::cout << chiave << '=' << valore << '\n';

    if (auto it = m.find("a"); it != m.end())      // init-statement (C++17)
        std::cout << it->second;
}
```

Esercizi consigliati su LeetCode

1 - Two Sum

26 - Remove Duplicates from Sorted Array

27 - Remove Element

412 - Fizz Buzz

1480 - Running Sum of 1d Array

Lezione 3.2 - Funzioni, overloading e lambda

Una funzione ha tipo di ritorno, nome e parametri. Passaggio per valore, per riferimento (`T&`), per riferimento costante (`const T&`). L'overloading permette piu' funzioni con lo stesso nome ma parametri diversi. Gli argomenti di default riducono la duplicazione.

```
#include <algorithm>
#include <vector>
int somma(int a, int b = 0) { return a + b; }
double somma(double a, double b) { return a + b; }

[[nodiscard]] constexpr int fattoriale(int n) {
    return n <= 1 ? 1 : n * fattoriale(n - 1);
}

int main() {
    std::vector<int> v{5, 2, 9, 1};
    int soglia = 3;
    auto grandi = std::count_if(v.begin(), v.end(), [soglia](int x) { return x > soglia; });
    auto quadrato = [](auto x) { return x * x; };   // lambda generica
    static_assert(fattoriale(5) == 120);
}
```

La ricorsione e' una tecnica fondamentale: ogni funzione ricorsiva ha un caso base e un passo ricorsivo.

Esercizi consigliati su LeetCode

509 - Fibonacci Number

70 - Climbing Stairs

50 - Pow(x, n)

344 - Reverse String

Capitolo 4 - Puntatori, riferimenti e memoria

Lezione 4.1 - Riferimenti e puntatori

Un riferimento e' un alias di un oggetto esistente. Un puntatore contiene un indirizzo e puo' essere riassegnato o nullo (`nullptr`). Usa riferimenti quando possibile, puntatori quando serve opzionalita' o riassegnazione.

```
int x = 10;
int& r = x;        // riferimento
int* p = &x;       // puntatore
*p = 20;           // x vale 20
r++;               // x vale 21
int arr[3] = {1, 2, 3};
int* q = arr;      // decadimento array -> puntatore
q[1] = 5;          // aritmetica dei puntatori: *(q+1)
```

Lezione 4.2 - Stack, heap, durata e RAII

Gli oggetti automatici vivono nello **stack** e sono distrutti all'uscita dallo scope. Gli oggetti dinamici vivono nell'**heap**. **RAII** (Resource Acquisition Is Initialization) lega la durata di una risorsa (memoria, file, mutex, socket) alla vita di un oggetto: il distruttore la rilascia automaticamente, anche in presenza di eccezioni.

Errori classici: memory leak, dangling pointer, double free, buffer overflow, use-after-free. Il C++ moderno li evita con gli smart pointer e i contenitori della STL.

```
#include <memory>
struct Nodo { int valore; std::unique_ptr<Nodo> next; };

auto a = std::make_unique<int>(42);        // proprieta' esclusiva
auto b = std::make_shared<int>(7);         // proprieta' condivisa (conteggio)
std::weak_ptr<int> w = b;                  // osservatore non proprietario
```

Nota

Regola d'oro del C++ moderno: non scrivere `new` e `delete` nel codice applicativo. Usa `make_unique`, `make_shared` e i contenitori.

Esercizi consigliati su LeetCode

141 - Linked List Cycle

206 - Reverse Linked List

21 - Merge Two Sorted Lists

876 - Middle of the Linked List

Capitolo 5 - Stringhe, I/O e file

Lezione 5.1 - std::string e std::string_view

`std::string` gestisce testo con memoria dinamica; `std::string_view` (C++17) e' una vista non proprietaria, ideale per parametri in sola lettura. Funzioni utili: `substr`, `find`, `replace`, `append`, `std::to_string`, `std::stoi`.

```
#include <string>
#include <string_view>
#include <format>      // C++20
#include <iostream>

int contaVocali(std::string_view s) {
    int n = 0;
    for (char c : s) if (std::string_view("aeiouAEIOU").find(c) != std::string_view::npos) ++n;
    return n;
}
int main() {
    std::string s = "Ciao mondo";
    std::cout << std::format("'{}' ha {} vocali\n", s, contaVocali(s));
}
```

Esercizi consigliati su LeetCode

14 - Longest Common Prefix

125 - Valid Palindrome

242 - Valid Anagram

387 - First Unique Character in a String

49 - Group Anagrams

5 - Longest Palindromic Substring

Lezione 5.2 - Stream e file

Le classi `std::ifstream`, `std::ofstream` e `std::fstream` leggono e scrivono file, chiudendoli automaticamente (RAII). C++17 aggiunge `std::filesystem` per percorsi e directory.

```
#include <fstream>
#include <filesystem>
#include <sstream>
#include <iostream>
namespace fs = std::filesystem;

int main() {
    { std::ofstream out("dati.txt"); out << "riga 1\nriga 2\n"; }
    std::ifstream in("dati.txt");
    std::string riga;
    while (std::getline(in, riga)) std::cout << riga << '\n';
    for (auto& e : fs::directory_iterator(".")) std::cout << e.path() << '\n';
}
```

Parte 2 - OOP, template e C++ moderno

Capitolo 6 - Programmazione orientata agli oggetti

Lezione 6.1 - Classi, costruttori e distruttori

Una classe incapsula dati e comportamento. `struct` e `class` differiscono solo per la visibilita' di default (pubblica vs privata). I costruttori inizializzano, il distruttore libera le risorse.

```
#include <string>
#include <utility>

class Conto {
public:
    Conto(std::string titolare, double saldo = 0.0)
        : titolare_(std::move(titolare)), saldo_(saldo) {}
    void versa(double v) { saldo_ += v; }
    [[nodiscard]] double saldo() const noexcept { return saldo_; }
private:
    std::string titolare_;
    double saldo_;
};
```

- Usa la **lista di inizializzazione** dei membri.
- Marca `const` i metodi che non modificano lo stato.
- `explicit` sui costruttori a un argomento evita conversioni implicite.

Lezione 6.2 - Regola del 5 / del 0, copia e move

Se una classe gestisce una risorsa servono: distruttore, costruttore di copia, assegnazione di copia, costruttore di move, assegnazione di move (**regola del 5**). Meglio ancora: usa membri che gestiscono la risorsa (`vector`, `unique_ptr`) e non scrivere nulla (**regola del 0**).

La **move semantics** (C++11) trasferisce le risorse da oggetti temporanei (rvalue) invece di copiarle: `std::move` converte in rvalue reference `T&&`.

```
class Buffer {
    std::size_t n_; int* d_;
public:
    explicit Buffer(std::size_t n) : n_(n), d_(new int[n]{}) {}
    ~Buffer() { delete[] d_; }
    Buffer(const Buffer& o) : n_(o.n_), d_(new int[o.n_]) { std::copy(o.d_, o.d_ + n_, d_); }
    Buffer& operator=(Buffer o) noexcept { std::swap(n_, o.n_); std::swap(d_, o.d_); return *this; }
    Buffer(Buffer&& o) noexcept : n_(std::exchange(o.n_, 0)), d_(std::exchange(o.d_, nullptr)) {}
};
```

Lezione 6.3 - Ereditarieta' e polimorfismo

Il polimorfismo a runtime usa funzioni `virtual`. La classe base deve avere distruttore virtuale. `override` e `final` rendono esplicite le intenzioni. Preferisci la composizione all'ereditarieta' quando possibile.

```
#include <memory>
#include <vector>
#include <numbers>
struct Forma {
    virtual ~Forma() = default;
    virtual double area() const = 0;   // classe astratta
};
struct Cerchio : Forma {
    double r;
    explicit Cerchio(double r) : r(r) {}
    double area() const override { return std::numbers::pi * r * r; }
};
struct Rettangolo : Forma {
    double w, h;
    Rettangolo(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};
int main() {
    std::vector<std::unique_ptr<Forma>> v;
    v.push_back(std::make_unique<Cerchio>(2));
    v.push_back(std::make_unique<Rettangolo>(3, 4));
    double tot = 0; for (auto& f : v) tot += f->area();
}
```

Lezione 6.4 - Overloading degli operatori, eccezioni, principi SOLID

Si possono ridefinire operatori (`+`, `==`, `<<`, `[]`, `()`) per tipi definiti dall'utente: fallo solo quando il significato e' naturale. Le eccezioni (`try`/`catch`/`throw`) segnalano errori eccezionali; con RAII garantiscono la pulizia. In alternativa: `std::optional`, `std::expected` (C++23), codici di errore.

I principi **SOLID** (Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion) guidano un design manutenibile. Pattern importanti: Strategy, Observer, Factory, Builder, Singleton (con cautela), Visitor, Decorator, Adapter.

Esercizi consigliati su LeetCode

146 - LRU Cache (design di classi)

155 - Min Stack

232 - Implement Queue using Stacks

225 - Implement Stack using Queues

705 - Design HashSet

706 - Design HashMap

208 - Implement Trie (Prefix Tree)

355 - Design Twitter

Capitolo 7 - Template e programmazione generica

Lezione 7.1 - Template di funzione e di classe

I template generano codice per tipi diversi a compile time senza overhead a runtime.

```
template <typename T>
T massimo(T a, T b) { return a < b ? b : a; }

template <typename T, std::size_t N>
class Array {
    T dati_[N]{};
public:
    constexpr T& operator[](std::size_t i) { return dati_[i]; }
    constexpr std::size_t size() const { return N; }
};

template <typename... Args>             // variadic template
auto somma(Args... args) { return (args + ...); }   // fold expression (C++17)
```

Lezione 7.2 - Concepts, SFINAE e metaprogrammazione

I **concepts** (C++20) esprimono vincoli sui parametri template con messaggi d'errore chiari, sostituendo il vecchio SFINAE. `if constexpr` (C++17) sceglie il ramo a compile time. `<type_traits>` offre `is_integral_v`, `enable_if`, `remove_reference_t`, ecc.

```
#include <concepts>
template <std::integral T>
T mcd(T a, T b) { return b == 0 ? a : mcd(b, a % b); }

template <typename T>
concept Stampabile = requires(T t, std::ostream& os) { os << t; };

template <Stampabile T>
void stampa(const T& x) { std::cout << x << '\n'; }

template <typename T>
auto descrivi(const T& v) {
    if constexpr (std::is_pointer_v<T>) return "puntatore";
    else return "valore";
}
```

Esercizi consigliati su LeetCode

703 - Kth Largest Element in a Stream

295 - Find Median from Data Stream

Capitolo 8 - Il C++ moderno: C++11, 14, 17, 20, 23

Lezione 8.1 - Novita' di C++11/14/17

- **C++11**: `auto`, range-for, lambda, `nullptr`, `move`, smart pointer, `constexpr`, `thread`, `unordered_map`, `enum class`, `override`, `= default`/`= delete`, inizializzazione uniforme.
- **C++14**: lambda generiche, `make_unique`, `decltype(auto)`, variable template, separatori numerici `1'000'000`.
- **C++17**: structured bindings, `std::optional`, `std::variant`, `std::string_view`, `std::filesystem`, `if constexpr`, fold expression, algoritmi paralleli, CTAD, `inline` variables.

```
#include <optional>
#include <variant>
#include <string>
std::optional<int> parseInt(const std::string& s) {
    try { return std::stoi(s); } catch (...) { return std::nullopt; }
}
using Valore = std::variant<int, double, std::string>;
struct Visitatore {
    void operator()(int i) const { /* ... */ }
    void operator()(double d) const { /* ... */ }
    void operator()(const std::string& s) const { /* ... */ }
};
// std::visit(Visitatore{}, valore);
```

Lezione 8.2 - Novita' di C++20/23: concepts, ranges, coroutine, moduli

- **Ranges**: pipeline componibili e pigre.
- **Coroutine**: `co_await`, `co_yield`, `co_return` per codice asincrono e generatori.
- **Moduli**: `import std;` sostituisce `#include` e velocizza la compilazione.
- `std::format`, `std::span`, `std::jthread`, `<=>`, `consteval`, `std::source_location`.
- **C++23**: `std::expected`, `std::print`, `std::mdspan`, `std::generator`, `deducing this`, `std::flat_map`.

```
#include <ranges>
#include <vector>
#include <print>   // C++23
namespace vw = std::views;
int main() {
    std::vector<int> v{1,2,3,4,5,6,7,8,9,10};
    auto r = v | vw::filter([](int x){ return x % 2 == 0; })
               | vw::transform([](int x){ return x * x; })
               | vw::take(3);
    for (int x : r) std::print("{} ", x);   // 4 16 36
}
```

Lezione 8.3 - Buone pratiche: C++ Core Guidelines

Segui le **C++ Core Guidelines** (Stroustrup e Sutter): `const` per default, parametri di ingresso per `const&` o per valore se piccoli, restituire per valore (grazie a copy elision/RVO), `noexcept` sulle funzioni di move, evitare le variabili globali, usare `std::span` invece di coppie puntatore+dimensione, `gsl::not_null`, `[[nodiscard]]`. Strumenti: clang-tidy (`modernize-*`, `cppcoreguidelines-*`), cppcheck, sanitizer.

Parte 3 - STL e strutture dati

Capitolo 9 - La Standard Template Library

Lezione 9.1 - Contenitori sequenziali

La STL si compone di **contenitori**, **iteratori** e **algoritmi**. Contenitori sequenziali: `std::array` (dimensione fissa), `std::vector` (array dinamico, il default), `std::deque`, `std::list` (lista doppia), `std::forward_list`.

Contenitore	Accesso	Inserimento in coda	Inserimento in mezzo
vector	O(1)	O(1) ammortizzato	O(n)
deque	O(1)	O(1)	O(n)
list	O(n)	O(1)	O(1) con iteratore

Nota pratica: `std::vector` e' quasi sempre piu' veloce di `std::list` grazie alla localita' di cache. Usa `reserve()` se conosci la dimensione e `emplace_back` per costruire in loco.

```
std::vector<int> v;
v.reserve(100);
for (int i = 0; i < 100; ++i) v.emplace_back(i);
v.erase(std::remove_if(v.begin(), v.end(), [](int x){ return x % 2; }), v.end()); // erase-remove
std::erase_if(v, [](int x){ return x > 50; });   // C++20
```

Esercizi consigliati su LeetCode

88 - Merge Sorted Array

283 - Move Zeroes

66 - Plus One

53 - Maximum Subarray

121 - Best Time to Buy and Sell Stock

238 - Product of Array Except Self

189 - Rotate Array

Lezione 9.2 - Contenitori associativi e hash

`std::map`/`std::set` (albero rosso-nero, O(log n), ordinati), `std::unordered_map`/`unordered_set` (tabella hash, O(1) medio), `multimap`, `multiset`. C++23 aggiunge `flat_map`/`flat_set`.

```
#include <unordered_map>
#include <string>
std::unordered_map<std::string, int> freq;
for (const std::string& w : parole) ++freq[w];
if (auto it = freq.find("ciao"); it != freq.end()) { /* trovato */ }
// chiave custom: servono std::hash e operator==
struct Punto { int x, y; bool operator==(const Punto&) const = default; };
template<> struct std::hash<Punto> {
    std::size_t operator()(const Punto& p) const noexcept { return std::hash<int>()(p.x) * 31 + std::hash<i
};
```

Esercizi consigliati su LeetCode

217 - Contains Duplicate

219 - Contains Duplicate II

347 - Top K Frequent Elements

560 - Subarray Sum Equals K

128 - Longest Consecutive Sequence

3 - Longest Substring Without Repeating Characters

4 - Median of Two Sorted Arrays

Lezione 9.3 - Adattatori, iteratori e algoritmi

Adattatori: `stack`, `queue`, `priority_queue`. Categorie di iteratori: input, forward, bidirectional, random access, contiguous. Algoritmi in `<algorithm>` e `<numeric>`: `sort`, `stable_sort`, `binary_search`, `lower_bound`, `find`, `accumulate`, `transform`, `partition`, `unique`, `reverse`, `rotate`, `next_permutation`, `iota`, `reduce`. In C++17 si possono eseguire in parallelo con `std::execution::par`.

```
#include <algorithm>
#include <numeric>
#include <queue>
std::vector<int> v{5, 3, 8, 1};
std::sort(v.begin(), v.end(), std::greater<>{});
int somma = std::accumulate(v.begin(), v.end(), 0);
std::priority_queue<int, std::vector<int>, std::greater<>> minHeap;
auto it = std::lower_bound(v.begin(), v.end(), 4);
```

Esercizi consigliati su LeetCode

215 - Kth Largest Element in an Array

1046 - Last Stone Weight

56 - Merge Intervals

75 - Sort Colors

31 - Next Permutation

912 - Sort an Array

Capitolo 10 - Strutture dati classiche (implementarle da zero)

Lezione 10.1 - Liste concatenate, pile e code

Implementare le strutture a mano insegna la gestione della memoria. Esempio: pila con lista singola e unique_ptr.

```
template <typename T>
class Pila {
    struct Nodo { T val; std::unique_ptr<Nodo> next; };
    std::unique_ptr<Nodo> testa_;
    std::size_t n_ = 0;
public:
    void push(T v) { testa_ = std::unique_ptr<Nodo>(new Nodo{std::move(v), std::move(testa_)}); ++n_; }
    T pop() { T v = std::move(testa_->val); testa_ = std::move(testa_->next); --n_; return v; }
    const T& top() const { return testa_->val; }
    bool empty() const { return n_ == 0; }
};
```

Temi: lista doppia, lista circolare, coda circolare (ring buffer), deque. Tecniche: due puntatori (lento/veloce), nodo sentinella, inversione in loco.

Esercizi consigliati su LeetCode

20 - Valid Parentheses

234 - Palindrome Linked List

19 - Remove Nth Node From End of List

160 - Intersection of Two Linked Lists

142 - Linked List Cycle II

2 - Add Two Numbers

23 - Merge k Sorted Lists

25 - Reverse Nodes in k-Group

739 - Daily Temperatures

84 - Largest Rectangle in Histogram

239 - Sliding Window Maximum

Lezione 10.2 - Alberi binari e di ricerca

Un albero binario ha nodi con al massimo due figli. Un BST mantiene `sinistro < nodo < destro`. Visite: pre-order, in-order, post-order, per livelli (BFS). Alberi bilanciati: AVL, rosso-nero (usato da `std::map`), B-tree (database e file system).

```
struct Nodo { int val; Nodo *sx = nullptr, *dx = nullptr; };

void inorder(const Nodo* n, std::vector<int>& out) {
    if (!n) return;
    inorder(n->sx, out); out.push_back(n->val); inorder(n->dx, out);
}
int altezza(const Nodo* n) { return n ? 1 + std::max(altezza(n->sx), altezza(n->dx)) : 0; }
```

Esercizi consigliati su LeetCode

104 - Maximum Depth of Binary Tree

94 - Binary Tree Inorder Traversal

102 - Binary Tree Level Order Traversal

226 - Invert Binary Tree

100 - Same Tree

101 - Symmetric Tree

98 - Validate Binary Search Tree

230 - Kth Smallest Element in a BST

235 - Lowest Common Ancestor of a BST

236 - Lowest Common Ancestor of a Binary Tree

105 - Construct Binary Tree from Preorder and Inorder Traversal

124 - Binary Tree Maximum Path Sum

297 - Serialize and Deserialize Binary Tree

Lezione 10.3 - Heap, tabelle hash, trie, union-find

- **Heap** (coda con priorita'): inserimento/estrazione O(log n); `std::priority_queue`, `make_heap`.
- **Tabella hash**: funzione di hash + gestione collisioni (concatenamento o indirizzamento aperto); fattore di carico e rehash.
- **Trie**: albero di prefissi per dizionari e autocompletamento.
- **Union-Find (DSU)**: insiemi disgiunti con path compression e union by rank, quasi O(1).
- **Segment tree / Fenwick tree**: query su intervalli in O(log n).
- **Skip list, Bloom filter, LRU cache**: strutture usate nei database e nei sistemi distribuiti.

```
struct DSU {
    std::vector<int> p, r;
    explicit DSU(int n) : p(n), r(n, 0) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unisci(int a, int b) {
        a = find(a); b = find(b); if (a == b) return false;
        if (r[a] < r[b]) std::swap(a, b);
        p[b] = a; if (r[a] == r[b]) ++r[a]; return true;
    }
};
```

Esercizi consigliati su LeetCode

684 - Redundant Connection

547 - Number of Provinces

721 - Accounts Merge

307 - Range Sum Query - Mutable

211 - Design Add and Search Words Data Structure

212 - Word Search II

460 - LFU Cache

295 - Find Median from Data Stream

Lezione 10.4 - Grafi

Rappresentazioni: matrice di adiacenza, liste di adiacenza (`vector<vector<int>>`), lista di archi. Visite: BFS (cammini minimi non pesati, livelli) e DFS (componenti connesse, cicli, ordinamento topologico).

```
std::vector<int> bfs(const std::vector<std::vector<int>>& g, int s) {
    std::vector<int> dist(g.size(), -1);
    std::queue<int> q; q.push(s); dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (dist[v] < 0) { dist[v] = dist[u] + 1; q.push(v); }
    }
    return dist;
}
```

Esercizi consigliati su LeetCode

200 - Number of Islands

133 - Clone Graph

207 - Course Schedule

210 - Course Schedule II

417 - Pacific Atlantic Water Flow

994 - Rotting Oranges

127 - Word Ladder

785 - Is Graph Bipartite?

261 - Graph Valid Tree

Parte 4 - Algoritmi principali

Capitolo 11 - Analisi e tecniche di base

Lezione 11.1 - Complessita' computazionale

La notazione Big-O descrive la crescita del tempo e dello spazio al crescere dell'input: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n), O(n!). Con n fino a 10^5 servono algoritmi O(n log n); con n fino a 20 e' accettabile O(2^n). Si analizzano caso migliore, medio, peggiore e costo ammortizzato (es. `push_back`).

Lezione 11.2 - Ricerca, two pointers, sliding window, prefix sum

Tecniche fondamentali sugli array: ricerca binaria (anche sulla risposta), due puntatori, finestra scorrevole, somme prefisse.

```
int ricercaBinaria(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;      // evita overflow
        if (a[mid] == x) return mid;
        (a[mid] < x ? lo : hi) = (a[mid] < x ? mid + 1 : mid - 1);
    }
    return -1;
}
```

Esercizi consigliati su LeetCode

704 - Binary Search

35 - Search Insert Position

33 - Search in Rotated Sorted Array

153 - Find Minimum in Rotated Sorted Array

875 - Koko Eating Bananas

167 - Two Sum II

15 - 3Sum

11 - Container With Most Water

42 - Trapping Rain Water

209 - Minimum Size Subarray Sum

424 - Longest Repeating Character Replacement

76 - Minimum Window Substring

438 - Find All Anagrams in a String

303 - Range Sum Query - Immutable

724 - Find Pivot Index

Capitolo 12 - Ordinamento

Lezione 12.1 - Algoritmi di ordinamento

Elementari O(n^2): bubble, selection, insertion (ottimo su piccoli input quasi ordinati). Efficienti O(n log n): merge sort (stabile), quicksort (in media il piu' veloce), heap sort. Lineari non basati su confronto: counting

sort, radix sort, bucket sort. `std::sort` e' un introsort (quick + heap + insertion).

```
void mergeSort(std::vector<int>& a, int l, int r, std::vector<int>& tmp) {
    if (r - l < 2) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m, tmp); mergeSort(a, m, r, tmp);
    std::merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, tmp.begin() + l);
    std::copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
}
```

Esercizi consigliati su LeetCode

912 - Sort an Array

148 - Sort List

169 - Majority Element

347 - Top K Frequent Elements

451 - Sort Characters By Frequency

164 - Maximum Gap (radix/bucket)

315 - Count of Smaller Numbers After Self

Capitolo 13 - Ricorsione, backtracking, greedy

Lezione 13.1 - Backtracking e divide et impera

Il backtracking esplora lo spazio delle soluzioni scartando i rami non validi: permutazioni, combinazioni, sottoinsiemi, N-regine, sudoku. Divide et impera: dividere, risolvere i sottoproblemi, combinare (merge sort, quicksort, Karatsuba).

```
void sottoinsiemi(const std::vector<int>& a, int i, std::vector<int>& cur, std::vector<std::vector<int>>& c
    if (i == (int)a.size()) { out.push_back(cur); return; }
    sottoinsiemi(a, i + 1, cur, out);                 // escludi
    cur.push_back(a[i]); sottoinsiemi(a, i + 1, cur, out); cur.pop_back(); // includi
}
```

Esercizi consigliati su LeetCode

78 - Subsets

90 - Subsets II

46 - Permutations

47 - Permutations II

39 - Combination Sum

40 - Combination Sum II

17 - Letter Combinations of a Phone Number

22 - Generate Parentheses

79 - Word Search

131 - Palindrome Partitioning

51 - N-Queens

37 - Sudoku Solver

Lezione 13.2 - Algoritmi greedy

Un algoritmo greedy compie la scelta localmente ottima sperando in un ottimo globale; va dimostrata la correttezza. Esempi: selezione di attivita', codifica di Huffman, monete (per alcuni sistemi), Kruskal e Prim.

Esercizi consigliati su LeetCode

455 - Assign Cookies

55 - Jump Game

45 - Jump Game II

134 - Gas Station

435 - Non-overlapping Intervals

452 - Minimum Number of Arrows to Burst Balloons

763 - Partition Labels

621 - Task Scheduler

135 - Candy

Capitolo 14 - Programmazione dinamica

Lezione 14.1 - Concetti e schemi classici

La DP risolve problemi con sottoproblemi sovrapposti e sottostruttura ottima, con memoizzazione (top-down) o tabulazione (bottom-up). Passi: definire lo stato, la ricorrenza, i casi base, l'ordine di calcolo, ottimizzare lo spazio.

```
int knapsack(const std::vector<int>& w, const std::vector<int>& v, int W) {
    std::vector<int> dp(W + 1, 0);
    for (std::size_t i = 0; i < w.size(); ++i)
        for (int c = W; c >= w[i]; --c) dp[c] = std::max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

Schemi: 1D (scalini, house robber), 2D su griglia, zaino 0/1 e illimitato, sequenze (LIS, LCS, edit distance), intervalli, bitmask, alberi, stringhe.

Esercizi consigliati su LeetCode

198 - House Robber

213 - House Robber II

322 - Coin Change

518 - Coin Change II

300 - Longest Increasing Subsequence

1143 - Longest Common Subsequence

72 - Edit Distance

62 - Unique Paths

64 - Minimum Path Sum

416 - Partition Equal Subset Sum

494 - Target Sum

139 - Word Break

152 - Maximum Product Subarray

91 - Decode Ways

312 - Burst Balloons

10 - Regular Expression Matching

5 - Longest Palindromic Substring

746 - Min Cost Climbing Stairs

Capitolo 15 - Algoritmi su grafi

Lezione 15.1 - Cammini minimi, ordinamento topologico, MST

- **Dijkstra**: cammini minimi con pesi non negativi, O((V+E) log V) con `priority_queue`.
- **Bellman-Ford**: pesi negativi e rilevamento di cicli negativi, O(VE).
- **Floyd-Warshall**: tutte le coppie, O(V^3).
- **Ordinamento topologico**: Kahn (BFS) o DFS su DAG.
- **MST**: Kruskal (con DSU) e Prim.
- **Altri**: componenti fortemente connesse (Tarjan, Kosaraju), ponti e punti di articolazione, flusso massimo (Dinic), A* per la ricerca di percorsi nei giochi.

```
using P = std::pair<long long, int>;
std::vector<long long> dijkstra(const std::vector<std::vector<std::pair<int,int>>>& g, int s) {
    std::vector<long long> d(g.size(), LLONG_MAX);
    std::priority_queue<P, std::vector<P>, std::greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;
        for (auto [v, w] : g[u]) if (du + w < d[v]) { d[v] = du + w; pq.push({d[v], v}); }
    }
    return d;
}
```

Esercizi consigliati su LeetCode

743 - Network Delay Time

787 - Cheapest Flights Within K Stops

1631 - Path With Minimum Effort

778 - Swim in Rising Water

1584 - Min Cost to Connect All Points

332 - Reconstruct Itinerary

269 - Alien Dictionary

802 - Find Eventual Safe States

1192 - Critical Connections in a Network

399 - Evaluate Division

695 - Max Area of Island

130 - Surrounded Regions

Capitolo 16 - Stringhe, matematica e bit manipulation

Lezione 16.1 - Algoritmi su stringhe

Pattern matching: KMP (prefix function), Z-algorithm, Rabin-Karp (rolling hash), Boyer-Moore; trie, suffix array, Aho-Corasick; algoritmo di Manacher per i palindromi.

Esercizi consigliati su LeetCode

28 - Find the Index of the First Occurrence in a String

459 - Repeated Substring Pattern

214 - Shortest Palindrome

647 - Palindromic Substrings

187 - Repeated DNA Sequences

336 - Palindrome Pairs

8 - String to Integer (atoi)

43 - Multiply Strings

Lezione 16.2 - Teoria dei numeri, geometria e operazioni sui bit

MCD (Euclide), crivello di Eratostene, esponenziazione modulare, inverso modulare, coefficienti binomiali, geometria computazionale (prodotto vettoriale, convex hull), operazioni sui bit (maschere, XOR trick, `std::popcount`, `std::bit_ceil`).

```
long long potMod(long long b, long long e, long long m) {
    long long r = 1; b %= m;
    for (; e; e >>= 1, b = b * b % m) if (e & 1) r = r * b % m;
    return r;
}
```

Esercizi consigliati su LeetCode

204 - Count Primes

202 - Happy Number

69 - Sqrt(x)

172 - Factorial Trailing Zeroes

268 - Missing Number

371 - Sum of Two Integers

149 - Max Points on a Line

1071 - Greatest Common Divisor of Strings

2 - Add Two Numbers

48 - Rotate Image

54 - Spiral Matrix

73 - Set Matrix Zeroes

Parte 5 - Concorrenza, prestazioni e librerie

Capitolo 17 - Concorrenza e parallelismo

Lezione 17.1 - Thread, mutex e variabili di condizione

Dal C++11 la libreria standard include `std::thread` (e `std::jthread` in C++20, che fa join automatico), `std::mutex`, `std::lock_guard`, `std::scoped_lock`, `std::condition_variable`, `std::atomic`, `std::future`, `std::async`. Una **data race** (accesso concorrente non sincronizzato con almeno una scrittura) e' comportamento indefinito.

```
#include <thread>
#include <mutex>
#include <vector>
#include <iostream>

int main() {
    long contatore = 0;
    std::mutex m;
    std::vector<std::jthread> ts;
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([&] {
            for (int k = 0; k < 100000; ++k) { std::scoped_lock lk(m); ++contatore; }
        });
    ts.clear();                       // join
    std::cout << contatore << '\n';   // 400000
}
```

Pattern: produttore/consumatore con coda bloccante, thread pool, `std::latch`/`std::barrier`/`std::semaphore` (C++20), `std::stop_token` per la cancellazione cooperativa. Per il lock-free: `std::atomic` e i memory order (relaxed, acquire, release, seq_cst).

Esercizi consigliati su LeetCode

1114 - Print in Order

1115 - Print FooBar Alternately

1116 - Print Zero Even Odd

1117 - Building H2O

1188 - Design Bounded Blocking Queue

1226 - The Dining Philosophers

1242 - Web Crawler Multithreaded

Lezione 17.2 - Async, coroutine e algoritmi paralleli

`std::async` e `std::future` offrono un modello a task. Le coroutine C++20 permettono di scrivere codice asincrono in stile sequenziale (usate da librerie come Boost.Asio e cppcoro). Gli algoritmi paralleli (`std::execution::par`) parallelizzano `sort`, `transform`, `reduce`. Librerie: Intel oneTBB, OpenMP, Taskflow, HPX; per la GPU CUDA, SYCL, OpenCL.

```
#include <future>
#include <numeric>
#include <execution>
auto f = std::async(std::launch::async, [] { return 6 * 7; });
int r = f.get();
double s = std::reduce(std::execution::par, v.begin(), v.end(), 0.0);
```

Capitolo 18 - Prestazioni e ottimizzazione

Lezione 18.1 - Misurare, profilare, ottimizzare

Prima misura, poi ottimizza. Strumenti: `perf`, Valgrind/Callgrind, Google Benchmark, Tracy, VTune, flamegraph. Principi: scegliere l'algoritmo giusto, migliorare la localita' di cache (struct of arrays, contenitori contigui), evitare allocazioni inutili (`reserve`, `string_view`, pool/arena allocator, `std::pmr`), usare SIMD (intrinsics, `std::simd`), ottimizzazioni del compilatore (`-O3`, LTO, PGO), `constexpr` per spostare calcoli a compile time.

Lezione 18.2 - Comportamento indefinito e sicurezza

Undefined behavior: overflow dei signed, accesso fuori limiti, dereferenziazione di nullptr, uso di oggetti distrutti, data race, violazione dello strict aliasing. Difese: sanitizer (ASan, UBSan, TSan, MSan), `at()` invece di `[]` quando serve, `std::span`, analisi statica, fuzzing (libFuzzer, AFL++), hardening del compilatore (`-D_GLIBCXX_ASSERTIONS`, `-fstack-protector`, CFI).

Capitolo 19 - Le principali librerie del C++

Lezione 19.1 - Librerie generali e utilita'

- **Boost**: Asio (rete/asincrono), Beast (HTTP/WebSocket), Spirit, Graph, Program_options, Filesystem, Multiprecision, Serialization.
- **Abseil** (Google): contenitori hash veloci, stringhe, tempo. **Folly** (Meta).
- **fmt / spdlog**: formattazione e logging (fmt e' alla base di `std::format`).
- **nlohmann/json, simdjson, RapidJSON**: JSON. **protobuf, FlatBuffers, Cap'n Proto**: serializzazione binaria.
- **CLI11, cxxopts**: argomenti da riga di comando. **Catch2, GoogleTest, doctest**: test.
- **Eigen, Armadillo, BLAS/LAPACK**: algebra lineare. **OpenCV**: visione artificiale. **Qt, wxWidgets, Dear ImGui**: interfacce grafiche.
- **SDL2, SFML, GLFW, Vulkan, OpenGL, DirectX**: grafica e multimedia.

Lezione 19.2 - Gestione delle dipendenze e build avanzato

Gestori di pacchetti: **vcpkg**, **Conan**, **CPM.cmake**/`FetchContent`. Concetti CMake moderni: target, `target_link_libraries`, `PUBLIC/PRIVATE/INTERFACE`, `find_package`, `install`, presets, toolchain file per la cross-compilazione. Altri sistemi: Meson, Bazel, Ninja. CI: GitHub Actions con matrice GCC/Clang/MSVC, sanitizer e clang-tidy.

```
include(FetchContent)
FetchContent_Declare(fmt GIT_REPOSITORY https://github.com/fmtlib/fmt GIT_TAG 10.2.1)
FetchContent_MakeAvailable(fmt)
target_link_libraries(corso PRIVATE fmt::fmt)
```

Lezione 19.3 - Test e qualita' del codice

Test unitari con GoogleTest/Catch2, mocking (gMock), TDD, code coverage (gcov/lcov), benchmark, fuzzing, analisi statica (clang-tidy, cppcheck, SonarQube), formattazione automatica con clang-format.

```
#include <catch2/catch_test_macros.hpp>
int somma(int a, int b) { return a + b; }
TEST_CASE("somma") { REQUIRE(somma(2, 3) == 5); }
```

Parte 6 - Programmazione web e mobile

Capitolo 20 - Programmazione web con C++

Lezione 20.1 - Il C++ nel backend web

Il C++ si usa sul web dove servono latenza minima e throughput elevato: server HTTP ad alte prestazioni, API gateway, motori di ricerca, servizi di trading, proxy (Envoy, NGINX moduli), browser engine. Framework e librerie principali: **Boost.Beast** (HTTP/WebSocket su Asio), **Crow**, **Drogon** (asincrono, ORM integrato), **Oat++**, **Pistache**, **cpp-httplib** (header-only, ottimo per iniziare), **Wt**, **gRPC** e **Thrift** per RPC.

```
#include "httplib.h"
#include <nlohmann/json.hpp>
using json = nlohmann::json;

int main() {
    httplib::Server svr;
    svr.Get("/api/saluto", [](const httplib::Request& req, httplib::Response& res) {
        std::string nome = req.has_param("nome") ? req.get_param_value("nome") : "mondo";
        res.set_content(json{{"messaggio", "Ciao " + nome}}.dump(), "application/json");
    });
    svr.Post("/api/somma", [](const httplib::Request& req, httplib::Response& res) {
        auto j = json::parse(req.body);
        res.set_content(json{{"risultato", j["a"].get<int>() + j["b"].get<int>()}}.dump(), "application/jso
    });
    svr.listen("0.0.0.0", 8080);
}
```

Concetti da conoscere: HTTP/1.1, HTTP/2, REST, JSON, routing, middleware, autenticazione (JWT, OAuth2), TLS (OpenSSL), CORS, rate limiting, logging strutturato, deploy con Docker.

Esercizi consigliati su LeetCode

146 - LRU Cache (cache di risposte)

362 - Design Hit Counter

359 - Logger Rate Limiter

535 - Encode and Decode TinyURL

588 - Design In-Memory File System

981 - Time Based Key-Value Store

Lezione 20.2 - Server asincroni con Boost.Asio e coroutine

Un server scalabile non usa un thread per connessione: usa un event loop (epoll/kqueue/IOCP) e I/O asincrono. Con le coroutine C++20 il codice asincrono ritorna lineare.

```
#include <boost/asio.hpp>
namespace asio = boost::asio;
using asio::ip::tcp;

asio::awaitable<void> sessione(tcp::socket s) {
    char buf[1024];
    for (;;) {
        auto n = co_await s.async_read_some(asio::buffer(buf), asio::use_awaitable);
        co_await asio::async_write(s, asio::buffer(buf, n), asio::use_awaitable);   // echo
    }
}
asio::awaitable<void> ascolta(unsigned short porta) {
    auto ex = co_await asio::this_coro::executor;
    tcp::acceptor acc(ex, {tcp::v4(), porta});
    for (;;) asio::co_spawn(ex, sessione(co_await acc.async_accept(asio::use_awaitable)), asio::detached);
}
int main() {
    asio::io_context io;
    asio::co_spawn(io, ascolta(9000), asio::detached);
    io.run();
}
```

Lezione 20.3 - WebAssembly: C++ nel browser

Con **Emscripten** si compila C++ in WebAssembly (WASM), eseguibile nei browser a velocita' quasi nativa. Casi d'uso: giochi, editor video/immagini, CAD, codec, porting di librerie esistenti (SQLite, OpenCV, FFmpeg). Binding con `embind`.

```
#include <emscripten/bind.h>
int fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
EMSCRIPTEN_BINDINGS(modulo) { emscripten::function("fib", &fib); }
```

```
emcc fib.cpp -O3 -lembind -o fib.js -sMODULARIZE -sEXPORT_NAME=Fib
```

Lezione 20.4 - Interfacce grafiche desktop e architettura full-stack

Per le GUI desktop: **Qt** (Widgets e QML, il framework C++ piu' completo, con segnali/slot), **Dear ImGui** (tool e debug UI), **wxWidgets**, **GTKmm**. Architettura tipica: frontend in JavaScript/TypeScript che comunica via REST/gRPC/WebSocket con un servizio C++ ad alte prestazioni; oppure moduli C++ esposti a Node.js (N-API), Python (pybind11, nanobind) e altri linguaggi tramite FFI.

```
#include <pybind11/pybind11.h>
int somma(int a, int b) { return a + b; }
PYBIND11_MODULE(esempio, m) { m.def("somma", &somma); }
```

Capitolo 21 - Programmazione mobile

Lezione 21.1 - C++ su Android (NDK, JNI)

Android permette di scrivere parti native in C++ con l'**NDK**: motori di gioco, elaborazione audio/video, crittografia, librerie multipiattaforma. La comunicazione con Kotlin/Java avviene via **JNI**. Il build usa CMake integrato in Gradle.

```
#include <jni.h>
#include <string>
extern "C" JNIEXPORT jstring JNICALL
Java_com_esempio_app_MainActivity_saluto(JNIEnv* env, jobject /*thiz*/) {
    std::string s = "Ciao da C++";
    return env->NewStringUTF(s.c_str());
}
```

```
// Kotlin
external fun saluto(): String
companion object { init { System.loadLibrary("nativo") } }
```

Punti chiave: ABI (arm64-v8a, armeabi-v7a, x86_64), gestione dei riferimenti locali/globali JNI, thread e `AttachCurrentThread`, `AAssetManager`, Vulkan/OpenGL ES, Oboe (audio a bassa latenza), logging con `__android_log_print`.

Lezione 21.2 - C++ su iOS (Objective-C++ e Swift interop)

Su iOS il C++ si integra con **Objective-C++** (file `.mm`) e, da Swift 5.9, con l'**interoperabilita' diretta Swift/C++**. Si usa per core condivisi tra piattaforme (logica, parser, motori), con interfaccia nativa in SwiftUI/UIKit. Build con Xcode o CMake (`-G Xcode`), Metal per la grafica.

```
// motore.hpp - core condiviso
class Motore { public: int calcola(int x) const { return x * 2; } };
```

```
// Swift 5.9+ con interop C++
import Motore
let m = Motore()
print(m.calcola(21))
```

Lezione 21.3 - Framework multipiattaforma: Qt/QML, Flutter, React Native

- **Qt/QML**: un'unica base di codice C++/QML per Android, iOS, desktop, embedded.
- **Flutter** e **React Native**: UI in Dart/JS con moduli nativi in C++ (FFI di Dart, TurboModules/JSI di React Native).
- **Kotlin Multiplatform**, **Djinni**, **SWIG**: generazione di binding.
- **Cocos2d-x**, **Unreal Engine**, **Godot (GDExtension)**: giochi mobili.

Linee guida architetturali: logica di business e algoritmi in C++ portabile (C++17/20, niente dipendenze di piattaforma), strato sottile di binding, UI nativa; test del core su desktop con CI, cross-compilazione con toolchain NDK/Xcode.

Esercizi consigliati su LeetCode

355 - Design Twitter (feed)

635 - Design Log Storage System

1166 - Design File System

380 - Insert Delete GetRandom O(1)

432 - All O`one Data Structure

Parte 7 - Videogiochi e grafica

Capitolo 22 - Fondamenti di game programming

Lezione 22.1 - Il game loop e l'architettura di un gioco

Il cuore di un videogioco e' il **game loop**: ripete lettura input, aggiornamento della simulazione, rendering. Per la stabilita' della fisica si usa un passo fisso (fixed timestep) disaccoppiato dal rendering.

```
#include <chrono>
using clock_t_ = std::chrono::steady_clock;

void gameLoop() {
    constexpr double dt = 1.0 / 60.0;
    double accumulatore = 0;
    auto precedente = clock_t_::now();
    bool inEsecuzione = true;
    while (inEsecuzione) {
        auto ora = clock_t_::now();
        accumulatore += std::chrono::duration<double>(ora - precedente).count();
        precedente = ora;
        processaInput(inEsecuzione);
        while (accumulatore >= dt) { aggiorna(dt); accumulatore -= dt; }
        disegna(accumulatore / dt);          // interpolazione
    }
}
```

Lezione 22.2 - Entity Component System (ECS) e gestione della memoria

L'architettura **ECS** separa dati (componenti), identita' (entita') e logica (sistemi), favorendo la cache e la composizione rispetto all'ereditarieta'. Librerie: **EnTT**, **flecs**. Tecniche: object pooling, allocatori ad arena/linear, data-oriented design (struct of arrays), spatial partitioning (griglia, quadtree, octree, BVH) per collisioni e culling.

```
#include <entt/entt.hpp>
struct Posizione { float x, y; };
struct Velocita { float dx, dy; };
int main() {
    entt::registry reg;
    auto e = reg.create();
    reg.emplace<Posizione>(e, 0.f, 0.f);
    reg.emplace<Velocita>(e, 1.f, 2.f);
    reg.view<Posizione, Velocita>().each([](auto& p, const auto& v) { p.x += v.dx; p.y += v.dy; });
}
```

Esercizi consigliati su LeetCode

1197 - Minimum Knight Moves

289 - Game of Life

1091 - Shortest Path in Binary Matrix

773 - Sliding Puzzle

529 - Minesweeper

79 - Word Search

782 - Transform to Chessboard

Lezione 22.3 - Matematica per i giochi: vettori, matrici, quaternioni

Servono: vettori (somma, prodotto scalare e vettoriale, normalizzazione), matrici 4x4 (traslazione, rotazione, scala, proiezione), quaternioni (rotazioni senza gimbal lock), interpolazione (lerp, slerp), collisioni (AABB, sfere, SAT, raycast). Librerie: **GLM**, **Eigen**, DirectXMath.

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
glm::mat4 model = glm::translate(glm::mat4(1.f), glm::vec3(1, 0, 0));
model = glm::rotate(model, glm::radians(45.f), glm::vec3(0, 1, 0));
glm::mat4 proj = glm::perspective(glm::radians(60.f), 16.f / 9.f, 0.1f, 100.f);
```

Capitolo 23 - Librerie e motori

Lezione 23.1 - SDL2, SFML e un primo gioco 2D

SDL2 e **SFML** forniscono finestra, input, audio e rendering 2D su Windows, Linux, macOS (e SDL anche su mobile e web).

```
#include <SFML/Graphics.hpp>
int main() {
    sf::RenderWindow w(sf::VideoMode({800, 600}), "Pong");
    w.setFramerateLimit(60);
    sf::CircleShape palla(10.f); palla.setPosition({400, 300});
    sf::Vector2f vel{4.f, 3.f};
    while (w.isOpen()) {
        while (auto ev = w.pollEvent()) if (ev->is<sf::Event::Closed>()) w.close();
        palla.move(vel);
        auto p = palla.getPosition();
        if (p.x < 0 || p.x > 780) vel.x = -vel.x;
        if (p.y < 0 || p.y > 580) vel.y = -vel.y;
        w.clear(); w.draw(palla); w.display();
    }
}
```

Progetti proposti: Pong, Snake, Tetris, Breakout, platform 2D con tilemap, shoot'em up, gioco a turni con IA.

Lezione 23.2 - Grafica 3D: OpenGL, Vulkan, DirectX, Metal

La pipeline grafica: vertex shader, rasterizzazione, fragment shader, depth test, blending. **OpenGL** (semplice, ottimo per imparare), **Vulkan** (basso livello, multipiattaforma, esplicito), **DirectX 12** (Windows/Xbox), **Metal** (Apple), **WebGPU**. Concetti: buffer (VBO/VAO/UBO), texture, framebuffer, illuminazione (Phong, PBR), shadow mapping, normal mapping, instancing, deferred rendering, compute shader. Librerie di supporto: GLFW, GLAD, stb_image, Assimp (caricamento modelli), bgfx, Dear ImGui.

```
const char* vs = R"(#version 330 core
layout(location=0) in vec3 pos;
uniform mat4 mvp;
void main() { gl_Position = mvp * vec4(pos, 1.0); })";
```

Lezione 23.3 - Unreal Engine, Godot e altri motori

Unreal Engine si programma in C++ (con Blueprint): `UCLASS`, `UPROPERTY`, `UFUNCTION`, Actor, Component, Gameplay Framework, garbage collector per gli `UObject`, replicazione di rete. **Godot** supporta C++ tramite GDExtension. Altri: **O3DE**, **Cocos2d-x**, **Urho3D**, **Ogre3D**, **raylib** (semplice e

didattico), **Box2D/Bullet/PhysX/Jolt** per la fisica, **FMOD/OpenAL/miniaudio** per l'audio.

```
UCLASS()
class AMioPersonaggio : public ACharacter {
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere, BlueprintReadWrite) float Salute = 100.f;
    UFUNCTION(BlueprintCallable) void SubisciDanno(float Danno);
};
```

Capitolo 24 - IA, rete e ottimizzazione nei giochi

Lezione 24.1 - Intelligenza artificiale per i giochi

Pathfinding (A*, navmesh, flow field, Dijkstra map), macchine a stati finiti, behavior tree, utility AI, GOAP, steering behaviors (seek, flee, flocking), minimax con potatura alpha-beta (scacchi, tris), Monte Carlo Tree Search, generazione procedurale (rumore di Perlin, WFC, dungeon generation).

Esercizi consigliati su LeetCode

1091 - Shortest Path in Binary Matrix

505 - The Maze II

490 - The Maze

913 - Cat and Mouse

464 - Can I Win

486 - Predict the Winner

877 - Stone Game

292 - Nim Game

348 - Design Tic-Tac-Toe

130 - Surrounded Regions

Lezione 24.2 - Multiplayer e ottimizzazione

Networking: UDP vs TCP, modelli client-server autoritativo, client-side prediction, riconciliazione, interpolazione delle entita', lag compensation, serializzazione compatta, librerie (ENet, GameNetworkingSockets, Photon). Ottimizzazione: profiling frame per frame, batching, culling (frustum, occlusion), LOD, streaming asset, multithreading con job system, evitare allocazioni durante il frame, SIMD.

Parte 8 - Sistemi embedded

Capitolo 25 - Fondamenti dell'embedded con C++

Lezione 25.1 - Hardware, microcontrollori e toolchain

Un sistema embedded e' un computer dedicato a un compito specifico, con risorse limitate (kB di RAM, MHz di CPU, bassi consumi). Famiglie: **ARM Cortex-M** (STM32, nRF52, RP2040), **AVR** (Arduino Uno), **ESP32** (Xtensa/RISC-V con Wi-Fi/BLE), **RISC-V**, PIC, MSP430. Toolchain: **arm-none-eabi-gcc**, PlatformIO, Zephyr, ESP-IDF, STM32CubeIDE, Pico SDK. Strumenti: debugger JTAG/SWD (OpenOCD, J-Link, ST-Link), oscilloscopio, analizzatore logico.

Cosa evitare/limitare in ambito embedded: eccezioni (`-fno-exceptions`), RTTI (`-fno-rtti`), allocazione dinamica e frammentazione dell'heap, iostream pesante. Cosa usare: `constexpr`, template, `std::array`, `std::span`, `std::optional`, `etl::` (Embedded Template Library), allocatori statici, `static_assert`.

Lezione 25.2 - Accesso ai registri e GPIO

I periferici sono mappati in memoria: si leggono e scrivono registri tramite puntatori `volatile`. `volatile` impedisce al compilatore di eliminare o riordinare gli accessi.

```
#include <cstdint>

struct GPIO {
    volatile std::uint32_t MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR;
};
inline GPIO& GPIOA = *reinterpret_cast<GPIO*>(0x40020000);

template <typename Port, unsigned Pin>
struct Led {
    static void init() { Port::MODER |= (1u << (Pin * 2)); }
    static void toggle() { Port::ODR ^= (1u << Pin); }
};
```

Un approccio C++ idiomatico incapsula i registri in classi con template a costo zero, trasformando errori di configurazione in errori di compilazione.

```
// Blink su Arduino
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() { digitalWrite(LED_BUILTIN, HIGH); delay(500); digitalWrite(LED_BUILTIN, LOW); delay(500); }
```

Lezione 25.3 - Interrupt, timer e vincoli real-time

Le **ISR** (Interrupt Service Routine) devono essere brevi: impostano flag o riempiono buffer, il lavoro pesante si fa nel main loop. Variabili condivise: `volatile` + sezioni critiche o `std::atomic`. Timer e PWM per temporizzazione e controllo motori; watchdog per il recupero da blocchi; modalita' a basso consumo (sleep). Real-time: determinismo, latenza massima, priorita', analisi WCET.

```
volatile bool tastoPremuto = false;
extern "C" void EXTI0_IRQHandler() { tastoPremuto = true; EXTI->PR |= 1; }
int main() {
    init();
    for (;;) { if (tastoPremuto) { tastoPremuto = false; gestisci(); } __WFI(); }
}
```

Esercizi consigliati su LeetCode

622 - Design Circular Queue (ring buffer)

641 - Design Circular Deque

346 - Moving Average from Data Stream

1472 - Design Browser History

191 - Number of 1 Bits

190 - Reverse Bits

461 - Hamming Distance

78 - Subsets (bitmask)

89 - Gray Code

1318 - Minimum Flips to Make a OR b Equal to c

Capitolo 26 - Protocolli, RTOS e sistemi connessi

Lezione 26.1 - Bus e protocolli: UART, I2C, SPI, CAN

- **UART**: seriale asincrona (debug, GPS, modem).
- **I2C**: bus a 2 fili con indirizzi (sensori, EEPROM, display).
- **SPI**: full-duplex veloce con chip select (memorie flash, display, SD).
- **CAN/CAN-FD**: bus robusto per automotive e industria.
- **1-Wire, USB, Ethernet, Modbus, LIN**.
- **Wireless**: BLE, Wi-Fi, LoRa, Zigbee, Thread, MQTT/CoAP a livello applicativo.

```
std::uint8_t leggiRegistroI2C(std::uint8_t addr, std::uint8_t reg) {
    i2c.start(); i2c.write(addr << 1); i2c.write(reg);
    i2c.restart(); i2c.write((addr << 1) | 1);
    auto v = i2c.read(/*nack*/true); i2c.stop();
    return v;
}
```

Driver ben progettati separano l'accesso all'hardware (HAL) dalla logica (pattern: interfacce astratte o concepts, iniezione delle dipendenze, test su PC con mock).

Lezione 26.2 - RTOS: FreeRTOS e Zephyr

Un **RTOS** offre task con priorita', scheduler preemptive, code, semafori, mutex (con priority inheritance), timer software, event groups. Problemi tipici: inversione di priorita', deadlock, overflow dello stack di un task, race condition tra ISR e task.

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

QueueHandle_t coda;
void taskSensore(void*) {
    for (;;) { int v = leggiSensore(); xQueueSend(coda, &v, portMAX_DELAY); vTaskDelay(pdMS_TO_TICKS(100)); }
}
void taskElabora(void*) {
    int v; for (;;) if (xQueueReceive(coda, &v, portMAX_DELAY)) elabora(v);
}
int main() {
    coda = xQueueCreate(8, sizeof(int));
    xTaskCreate(taskSensore, "sens", 256, nullptr, 2, nullptr);
    xTaskCreate(taskElabora, "elab", 256, nullptr, 1, nullptr);
    vTaskStartScheduler();
}
```

Lezione 26.3 - Bootloader, OTA, sicurezza e Embedded Linux

Bootloader (MCUboot, U-Boot), aggiornamenti OTA firmati, secure boot, crittografia (mbedTLS), gestione di flash e wear leveling, diagnostica e log in-field. Per sistemi piu' potenti si usa **Embedded Linux** (Yocto, Buildroot, Raspberry Pi): accesso a GPIO via `libgpiod`, `/dev/i2c`, `/dev/spidev`, driver in userspace, systemd, cross-compilazione con CMake toolchain file. Standard di codifica per sistemi critici: **MISRA C++**, **AUTOSAR C++14**, **CERT C++**, ISO 26262 (automotive), DO-178C (aerospaziale), IEC 61508.

Lezione 26.4 - Progetti pratici embedded

- LED lampeggiante con timer e interrupt (senza `delay`).
- Lettura di un sensore di temperatura I2C con filtro a media mobile e display OLED.
- Macchina a stati per un distributore (stati, eventi, transizioni).
- Datalogger su scheda SD con FreeRTOS.
- Stazione meteo ESP32 che pubblica su MQTT.
- Controllo PID di un motore DC con encoder.
- Driver SPI generico basato su template e concepts, testato su PC.

Esercizi consigliati su LeetCode

232 - Implement Queue using Stacks

1670 - Design Front Middle Back Queue

295 - Find Median from Data Stream

480 - Sliding Window Median

362 - Design Hit Counter

1429 - First Unique Number

Parte 9 - Sistemi operativi

Capitolo 27 - Programmazione di sistema (POSIX/Linux e Windows)

Lezione 27.1 - Processi, fork/exec, segnali

Un sistema operativo gestisce processi, memoria, file, dispositivi e rete. In C++ si usano le API di sistema: POSIX su Linux/macOS, Win32 su Windows. Un processo ha un PID, uno spazio di indirizzi, file descriptor. `fork()` clona il processo, `exec*()` ne sostituisce l'immagine, `waitpid()` attende il figlio.

```
#include <unistd.h>
#include <sys/wait.h>
#include <csignal>
#include <iostream>

int main() {
    pid_t pid = fork();
    if (pid == 0) { execlp("ls", "ls", "-l", nullptr); _exit(127); }
    int stato; waitpid(pid, &stato, 0);
    std::cout << "figlio terminato con " << WEXITSTATUS(stato) << '\n';
}
```

Segnali: `SIGINT`, `SIGTERM`, `SIGSEGV`, `SIGCHLD`; con `sigaction` si installano handler (solo funzioni async-signal-safe al loro interno).

Lezione 27.2 - File descriptor, pipe, IPC e memoria condivisa

Tutto e' un file: file regolari, pipe, socket, dispositivi. API: `open`, `read`, `write`, `close`, `dup2`, `pipe`, `mmap`, `fcntl`. IPC: pipe e FIFO, socket Unix, code di messaggi, memoria condivisa (`shm_open` + `mmap`), semafori, segnali. Wrapper RAII: classe `FileDescriptor` che chiama `close()` nel distruttore.

```
class Fd {
    int fd_;
public:
    explicit Fd(int fd = -1) : fd_(fd) {}
    ~Fd() { if (fd_ >= 0) ::close(fd_); }
    Fd(Fd&& o) noexcept : fd_(std::exchange(o.fd_, -1)) {}
    Fd& operator=(Fd&& o) noexcept { std::swap(fd_, o.fd_); return *this; }
    Fd(const Fd&) = delete; Fd& operator=(const Fd&) = delete;
    int get() const { return fd_; }
};
```

Lezione 27.3 - Una mini shell

Esercizio completo: leggi una riga, tokenizza, `fork`+`exec`, gestisci pipe (`a | b`), redirezioni (`>`, `<`), comandi interni (`cd`, `exit`), job in background (`&`) e `SIGINT`.

Esercizi consigliati su LeetCode

71 - Simplify Path

588 - Design In-Memory File System

609 - Find Duplicate File in System

1166 - Design File System

636 - Exclusive Time of Functions

862 - Shortest Subarray with Sum at Least K

Capitolo 28 - Concetti di sistemi operativi

Lezione 28.1 - Scheduling, memoria virtuale, paginazione

Scheduling della CPU: FCFS, SJF, Round Robin, priorita', MLFQ, CFS di Linux. Memoria virtuale: tabelle delle pagine, TLB, page fault, swapping, algoritmi di sostituzione (FIFO, LRU, Clock). Allocatori: first-fit/best-fit, buddy system, slab; `malloc` implementato con liste libere. Implementare un allocatore e' un ottimo esercizio.

Esercizi consigliati su LeetCode

146 - LRU Cache

460 - LFU Cache

621 - Task Scheduler

253 - Meeting Rooms II

56 - Merge Intervals

1834 - Single-Threaded CPU

1962 - Remove Stones to Minimize the Total

358 - Rearrange String k Distance Apart

Lezione 28.2 - Sincronizzazione e deadlock

Race condition, sezioni critiche, mutex, semafori, monitor, spinlock, lock-free. Problemi classici: produttore-consumatore, lettori-scrittori, filosofi a cena. Deadlock: condizioni di Coffman (mutua esclusione, hold and wait, no preemption, attesa circolare); prevenzione con ordinamento delle risorse (`std::scoped_lock`/`std::lock` acquisiscono piu' mutex senza deadlock).

Esercizi consigliati su LeetCode

1226 - The Dining Philosophers

1195 - Fizz Buzz Multithreaded

1279 - Traffic Light Controlled Intersection

207 - Course Schedule (rilevare cicli di dipendenza)

Lezione 28.3 - File system e I/O

Struttura di un file system: superblocco, inode, blocchi dati, directory, journaling (ext4), copy-on-write (Btrfs, ZFS, APFS), FAT. Operazioni di I/O: buffered vs direct, `mmap`, `io_uring`, `epoll`, I/O asincrono. Implementa un mini file system in un file immagine.

Capitolo 29 - Costruire un sistema operativo

Lezione 29.1 - Boot, bare metal e kernel minimale

Per sviluppare un OS in C++ si lavora in ambiente **freestanding**: niente libreria standard completa, niente eccezioni/RTTI (o implementati da te), cross-compiler (`x86_64-elf-g++` o `riscv64-elf-g++`),

emulatore **QEMU**. Sequenza di avvio: firmware (BIOS/UEFI) -> bootloader (GRUB, limine) -> kernel. Tappe di un kernel didattico:

- Boot con header Multiboot2, passaggio in modalita' 64 bit.
- Output su console VGA/seriale.
- GDT, IDT, gestione delle eccezioni e degli interrupt (PIC/APIC), timer PIT/HPET.
- Gestore di memoria fisica (bitmap/buddy) e virtuale (paginazione), heap del kernel.
- Tastiera (PS/2), driver ATA/AHCI/NVMe, file system.
- Scheduler cooperativo poi preemptive, context switch in assembly.
- Syscall (`syscall`/`sysret`), modalita' utente, caricamento di eseguibili ELF.
- Shell in user space, libc minimale, rete (driver e1000 + TCP/IP).

```
extern "C" void kmain(std::uint32_t magic, void* info) {
    Console::pulisci();
    Console::stampa("Kernel avviato\n");
    Gdt::init(); Idt::init(); Pic::init(); Timer::avvia(100);
    Paging::init(info); Heap::init();
    asm volatile("sti");
    for (;;) asm volatile("hlt");
}
```

```
// global constructors e operatori new nel kernel
void* operator new(std::size_t n) { return Heap::alloca(n); }
void operator delete(void* p) noexcept { Heap::libera(p); }
```

Risorse: OSDev wiki, "Operating Systems: Three Easy Pieces", xv6 (MIT), "Writing an OS in Rust/C++" tutorial, SerenityOS (un OS completo in C++ moderno, ottimo da leggere), Linux kernel (C, con Rust), Haiku e Fuchsia/Zircon (C++).

Lezione 29.2 - Driver, moduli e C++ nel kernel

Scrivere driver per Windows (WDK, KMDF) o per Linux (moduli in C; C++ non e' ammesso nel kernel Linux upstream). In user space con **FUSE** puoi implementare file system, con **libusb** driver USB, con **DPDK** e **io_uring** I/O ad alte prestazioni. Concetti: MMIO, DMA, interrupt handler top/bottom half, concorrenza nel kernel, sicurezza (ring 0 / ring 3, ASLR, NX, SMEP).

Esercizi consigliati su LeetCode

146 - LRU Cache

380 - Insert Delete GetRandom O(1)

705 - Design HashSet

622 - Design Circular Queue

1396 - Design Underground System

1348 - Tweet Counts Per Frequency

Parte 10 - Programmazione di reti

Capitolo 30 - Fondamenti di networking

Lezione 30.1 - Modello ISO/OSI e TCP/IP

Livelli TCP/IP: collegamento (Ethernet, Wi-Fi), rete (IP, ICMP, ARP), trasporto (TCP, UDP, QUIC), applicazione (HTTP, DNS, TLS, SMTP, SSH, MQTT). Concetti: indirizzi IPv4/IPv6, subnet, porte, NAT, routing, MTU, handshake a tre vie di TCP, controllo di flusso e di congestione. Strumenti: `ping`, `traceroute`, `ss`, `nc`, `tcpdump`, **Wireshark**, `curl`, `nmap`.

Endianness: la rete usa **big endian** (network byte order). Funzioni `htons`, `htonl`, `ntohs`, `ntohl`, oppure `std::byteswap` (C++23) e `std::endian` (C++20).

Lezione 30.2 - Socket BSD in C++

Server TCP sequenziale con socket POSIX:

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <iostream>

int main() {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1; setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt);
    sockaddr_in a{}; a.sin_family = AF_INET; a.sin_addr.s_addr = INADDR_ANY; a.sin_port = htons(9000);
    bind(srv, (sockaddr*)&a, sizeof a);
    listen(srv, 16);
    for (;;) {
        int c = accept(srv, nullptr, nullptr);
        char buf[1024];
        ssize_t n;
        while ((n = recv(c, buf, sizeof buf, 0)) > 0) send(c, buf, n, 0);   // echo
        close(c);
    }
}
```

Client TCP: `socket` -> `getaddrinfo` -> `connect` -> `send/recv` -> `close`. Per UDP: `SOCK_DGRAM`, `sendto/recvfrom`. Gestire sempre: letture/scritture parziali, `EINTR`, timeout, chiusura ordinata (`shutdown`), `SIGPIPE`.

Capitolo 31 - Server scalabili

Lezione 31.1 - Modelli di concorrenza dei server

1. **Un processo/thread per connessione**: semplice, non scala. 2. **Thread pool** con coda di lavori. 3. **Multiplexing** con `select`/`poll` e soprattutto `epoll` (Linux), `kqueue` (BSD/macOS), IOCP (Windows), `io_uring` (Linux moderno). 4. **Reactor** (event loop) o **Proactor** (completamento asincrono). 5. **Coroutine** sopra un event loop (Asio, libuv, liburing).

Il problema C10K/C10M: gestire decine di migliaia di connessioni con pochi thread, buffer non bloccanti, backpressure e timeout.

```
int ep = epoll_create1(0);
epoll_event ev{}; ev.events = EPOLLIN; ev.data.fd = srv;
epoll_ctl(ep, EPOLL_CTL_ADD, srv, &ev);
epoll_event eventi[64];
for (;;) {
    int n = epoll_wait(ep, eventi, 64, -1);
    for (int i = 0; i < n; ++i) {
        int fd = eventi[i].data.fd;
        if (fd == srv) { int c = accept4(srv, nullptr, nullptr, SOCK_NONBLOCK); /* registra c */ }
        else gestisciLettura(fd);
    }
}
```

Lezione 31.2 - Protocolli applicativi e serializzazione

Progettare un protocollo: framing (prefisso di lunghezza, delimitatori), versioning, checksum, gestione di messaggi parziali. Formati: JSON, Protocol Buffers, FlatBuffers, MessagePack, CBOR. Il codice di parsing deve essere robusto contro input malevoli (limiti di lunghezza, validazione, fuzzing).

```
struct Intestazione { std::uint32_t lunghezza; std::uint16_t tipo; std::uint16_t versione; };
// invio: lunghezza in network byte order, poi payload
```

Lezione 31.3 - TLS, sicurezza e RPC

TLS con OpenSSL, BoringSSL, mbedTLS, wolfSSL o `Boost.Asio::ssl`: certificati, catena di fiducia, SNI, ALPN, verifica dell'hostname. Evitare di scrivere crittografia in proprio. Altri temi: autenticazione, rate limiting, SYN flood, buffer overflow nei parser, DoS. **gRPC** (HTTP/2 + protobuf), **WebSocket**, **QUIC/HTTP3** (msquic, ngtcp2), **ZeroMQ**, **nanomsg** per messaggistica distribuita.

Esercizi consigliati su LeetCode

535 - Encode and Decode TinyURL

271 - Encode and Decode Strings (framing)

359 - Logger Rate Limiter

362 - Design Hit Counter

1146 - Snapshot Array

1244 - Design A Leaderboard

981 - Time Based Key-Value Store

1396 - Design Underground System

1500 - Design a File Sharing System

Capitolo 32 - Algoritmi e progetti di rete

Lezione 32.1 - Algoritmi di rete

Instradamento (Dijkstra e Bellman-Ford sono alla base di OSPF e RIP), spanning tree (STP), flusso massimo, load balancing (round robin, least connections, consistent hashing), rate limiting (token bucket, leaky bucket, sliding window), controllo di congestione (AIMD, CUBIC, BBR), compressione (Huffman, LZ77), checksum e CRC, codici di correzione degli errori.

Esercizi consigliati su LeetCode

743 - Network Delay Time
1334 - Find the City With the Smallest Number of Neighbors at a Threshold Distance
1135 - Connecting Cities With Minimum Cost
1319 - Number of Operations to Make Network Connected
1202 - Smallest String With Swaps
886 - Possible Bipartition
1584 - Min Cost to Connect All Points
1976 - Number of Ways to Arrive at Destination
2092 - Find All People With Secret

Lezione 32.2 - Progetti di rete

- Chat multiutente con epoll o Asio (stanze, nickname, comandi).
- Client/server HTTP minimale con parser delle richieste e file statici.
- Port scanner e traceroute con raw socket (solo su reti tue!).
- Proxy TCP/SOCKS5 e load balancer.
- Server DNS ricorsivo semplice con cache.
- Trasferimento file affidabile sopra UDP (ritrasmissioni, finestra scorrevole).
- Client MQTT e broker minimale.
- Gioco multiplayer client-server autoritativo (vedi Parte 7).

Parte 11 - Programmazione di database

Capitolo 33 - Usare database da C++

Lezione 33.1 - SQL e SQLite

Un database relazionale organizza i dati in tabelle; SQL permette di interrogarli (`SELECT`, `JOIN`, `GROUP BY`), modificarli (`INSERT`, `UPDATE`, `DELETE`) e definirne lo schema. Concetti: chiavi primarie/esterne, indici, normalizzazione, transazioni e proprieta' **ACID**, livelli di isolamento (read committed, repeatable read, serializable), query plan (`EXPLAIN`).

SQLite e' la via piu' semplice per cominciare (libreria C embedded, zero configurazione).

```
#include <sqlite3.h>
#include <iostream>
#include <memory>

int main() {
    sqlite3* raw = nullptr;
    sqlite3_open("corso.db", &raw);
    std::unique_ptr<sqlite3, decltype(&sqlite3_close)> db(raw, &sqlite3_close);

    sqlite3_exec(db.get(), "CREATE TABLE IF NOT EXISTS utenti(id INTEGER PRIMARY KEY, nome TEXT, eta INT);"

    sqlite3_stmt* st;
    sqlite3_prepare_v2(db.get(), "INSERT INTO utenti(nome, eta) VALUES(?, ?);", -1, &st, nullptr);
    sqlite3_bind_text(st, 1, "Anna", -1, SQLITE_TRANSIENT);   // query parametrizzata: niente SQL injection
    sqlite3_bind_int(st, 2, 30);
    sqlite3_step(st); sqlite3_finalize(st);

    sqlite3_prepare_v2(db.get(), "SELECT nome, eta FROM utenti;", -1, &st, nullptr);
    while (sqlite3_step(st) == SQLITE_ROW)
        std::cout << sqlite3_column_text(st, 0) << ' ' << sqlite3_column_int(st, 1) << '\n';
    sqlite3_finalize(st);
}
```

> **Nota**
>
> Usa sempre query parametrizzate (prepared statement): concatenare stringhe nelle query espone ad attacchi di SQL injection.

> **Esercizi consigliati su LeetCode**
>
> 175 - Combine Two Tables (SQL)
>
> 176 - Second Highest Salary (SQL)
>
> 181 - Employees Earning More Than Their Managers (SQL)
>
> 182 - Duplicate Emails (SQL)
>
> 183 - Customers Who Never Order (SQL)
>
> 184 - Department Highest Salary (SQL)
>
> 262 - Trips and Users (SQL)
>
> 185 - Department Top Three Salaries (SQL)

Lezione 33.2 - PostgreSQL, MySQL e ORM

Client C++ per i principali DBMS: **libpqxx** (PostgreSQL), **MySQL Connector/C++** e **MariaDB Connector**, **ODBC/nanodbc**, **SOCI** (astrazione multi-database), **sqlpp11**, **ODB**, **Drogon ORM**, **QxOrm**; per NoSQL: **mongocxx** (MongoDB), **hiredis/redis-plus-plus** (Redis), **cassandra-cpp-driver**, **cpp-driver per Couchbase**. Pool di connessioni, transazioni RAII, gestione degli errori, mapping tra tipi SQL e C++ (`std::optional` per NULL).

```
#include <pqxx/pqxx>
int main() {
    pqxx::connection c("dbname=corso user=app host=localhost");
    pqxx::work tx(c);
    pqxx::result r = tx.exec_params("SELECT nome FROM utenti WHERE eta > $1", 18);
    for (auto riga : r) std::cout << riga[0].as<std::string>() << '\n';
    tx.exec_params("UPDATE utenti SET eta = eta + 1 WHERE id = $1", 7);
    tx.commit();    // senza commit: rollback automatico nel distruttore
}
```

Capitolo 34 - Costruire un database: strutture e algoritmi

Lezione 34.1 - Storage engine: pagine, B-tree, LSM-tree

Un DBMS e' fatto di: parser SQL, planner/ottimizzatore, executor, **storage engine**, buffer pool, gestore di transazioni, log (WAL). Molti database sono scritti in C++: MySQL, MongoDB, ClickHouse, RocksDB, LevelDB, DuckDB, ScyllaDB, FoundationDB.

- **Pagine** di dimensione fissa (4-16 KB) su disco; **buffer pool** con politica LRU/Clock.
- **B+tree**: indice ordinato con letture ottimizzate per disco; usato da InnoDB e PostgreSQL.
- **LSM-tree**: memtable in RAM + SSTable su disco con compaction (RocksDB, Cassandra); ottimo per scritture.
- **Hash index**, **bitmap index**, **R-tree** (geospaziale), **inverted index** (full-text).
- **WAL** (Write-Ahead Logging) per durabilita' e recupero dai crash (ARIES).
- **MVCC** per letture non bloccanti; lock a due fasi per l'isolamento.
- **Colonnare** vs **riga**: analitica (OLAP) vs transazionale (OLTP).

```
// Scheletro di una pagina slotted
struct Pagina {
    static constexpr std::size_t DIM = 4096;
    std::uint16_t nSlot, offsetLibero;
    std::byte dati[DIM - 4];
};
```

Esercizi consigliati su LeetCode

146 - LRU Cache (buffer pool)

460 - LFU Cache

981 - Time Based Key-Value Store

1146 - Snapshot Array (MVCC)

355 - Design Twitter

981 - Time Based Key-Value Store

706 - Design HashMap

208 - Implement Trie (Prefix Tree)

307 - Range Sum Query - Mutable

315 - Count of Smaller Numbers After Self

23 - Merge k Sorted Lists (merge delle SSTable)

Lezione 34.2 - Parser e ottimizzatore di query

Un parser SQL si costruisce con lexer + parser a discesa ricorsiva (o ANTLR, Bison, PEGTL). Il risultato e' un AST che il planner trasforma in un piano logico e poi fisico (scelta di indici, ordine dei join, algoritmi nested-loop/hash/merge join). Un'ottimizzazione basata sui costi usa statistiche e istogrammi.

```
struct Espr { virtual ~Espr() = default; virtual double valuta() const = 0; };
// Parser a discesa ricorsiva per: expr := term (('+'|'-') term)*
```

Esercizi consigliati su LeetCode

224 - Basic Calculator

227 - Basic Calculator II

772 - Basic Calculator III

150 - Evaluate Reverse Polish Notation

394 - Decode String

726 - Number of Atoms

1096 - Brace Expansion II

Lezione 34.3 - Progetti con database

- Rubrica / gestionale con SQLite e interfaccia a riga di comando.
- Key-value store persistente con WAL e compattazione (stile Bitcask).
- Piccolo B-tree su file con pagine e buffer pool.
- Mini SQL engine: CREATE/INSERT/SELECT con WHERE su tabelle in memoria, poi su disco.
- API REST (Parte 6) collegata a PostgreSQL con pool di connessioni.
- Cache distribuita con protocollo Redis (RESP) sopra il server epoll della Parte 10.

Parte 12 - Percorso di studio e progetti finali

Capitolo 35 - Piano di studio e preparazione ai colloqui

Lezione 35.1 - Piano di studio suggerito (24 settimane)

- Settimane 1-4: Parte 1 (basi) + 30 problemi LeetCode Easy su array, stringhe, hash map.
- Settimane 5-8: Parte 2 (OOP, template, C++ moderno) + liste concatenate, stack, queue.
- Settimane 9-12: Parte 3 (STL e strutture dati) + alberi, heap, grafi (Medium).
- Settimane 13-16: Parte 4 (algoritmi): ricerca binaria, backtracking, DP, grafi.
- Settimane 17-18: Parte 5 (concorrenza, prestazioni, librerie, test).
- Settimane 19-24: scegli un percorso di specializzazione (Parti 6-11) e realizza un progetto completo.

Tecnica per LeetCode: leggi il problema, scrivi esempi, individua il pattern (hash map, two pointers, sliding window, stack, binary search, BFS/DFS, DP, heap, backtracking, union-find), stima la complessita', implementa, testa casi limite. Liste utili: **Blind 75**, **NeetCode 150**, **Grind 75**. Obiettivo: 150-300 problemi risolti in modo ragionato, non 1000 a memoria.

> **Esercizi consigliati su LeetCode**
>
> Blind 75 - lista curata di problemi fondamentali
>
> NeetCode 150 - percorso per pattern
>
> Grind 75 - piano di studio a settimane
>
> LeetCode Explore Cards - "Arrays 101", "Linked List", "Binary Tree", "Graph", "Dynamic Programming"

Lezione 35.2 - Progetti finali consigliati

1. **Libreria di strutture dati** generiche con concepts e test (Catch2). 2. **Server HTTP** asincrono con Asio e database PostgreSQL. 3. **Gioco 2D** con ECS (EnTT) e SDL/SFML, poi port su WebAssembly. 4. **Firmware** per ESP32/STM32 con FreeRTOS, sensore e MQTT. 5. **Mini kernel** x86_64/RISC-V che stampa su seriale, gestisce interrupt e memoria. 6. **Key-value store** persistente con protocollo di rete. 7. **App mobile** con core C++ condiviso tra Android (JNI) e iOS (Swift interop).

Lezione 35.3 - Risorse per continuare

- Libri: "A Tour of C++" (Stroustrup), "Effective Modern C++" (Meyers), "C++ Concurrency in Action" (Williams), "C++ Templates: The Complete Guide", "The C++ Programming Language", "Programming: Principles and Practice Using C++", "Game Programming Patterns", "Computer Systems: A Programmer's Perspective", "Designing Data-Intensive Applications", "Introduction to Algorithms (CLRS)".
- Siti: cppreference.com, isocpp.org, C++ Core Guidelines, Compiler Explorer (godbolt.org), learncpp.com, cp-algorithms.com.
- Conferenze e canali: CppCon, C++Now, Meeting C++, The Cherno, Jason Turner (C++ Weekly).
- Pratica: Exercism, Codeforces, AtCoder, Advent of Code, contributi open source (LLVM, Qt, Boost, SerenityOS).